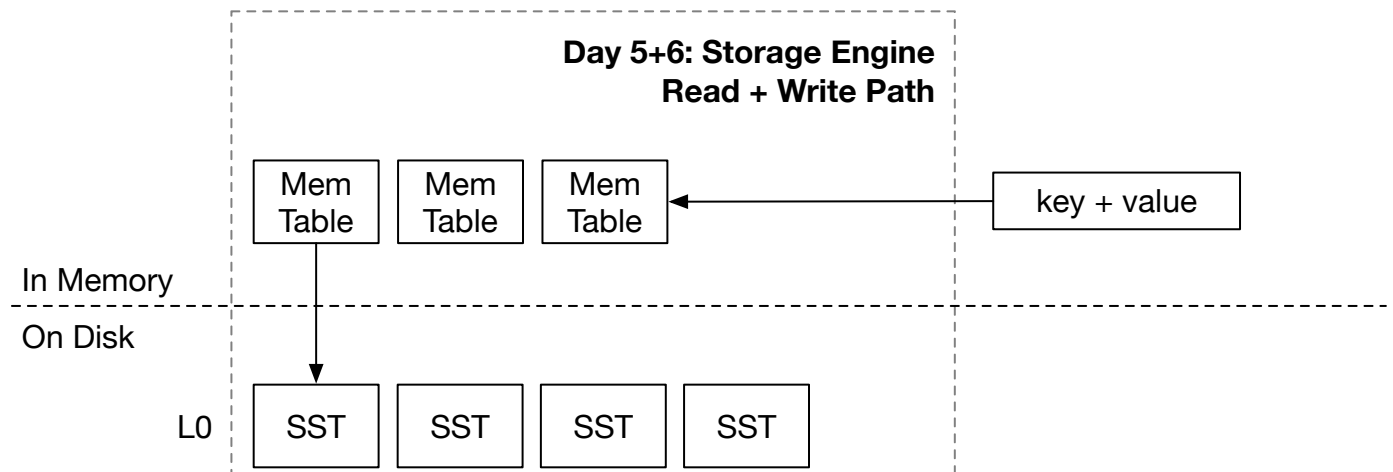


Write Path



In this chapter, you will:

- Implement the LSM write path with L0 flush.
- Implement the logic to correctly update the LSM state.

To copy the test cases into the starter code and run them,

```
cargo x copy-test --week 1 --day 6
cargo x scheck
```

Task 1: Flush Memtable to SST

At this point, we have all in-memory things and on-disk files ready, and the storage engine is able to read and merge the data from all these structures. Now, we are going to implement the logic to move things from memory to the disk (so-called flush), and complete the Mini-LSM week 1 course.

In this task, you will need to modify:

```
src/lsm_storage.rs
src/mem_table.rs
```

You will need to modify `LSMStorageInner::force_flush_next_imm_memtable` and `MemTable::flush`. In `LSMStorageInner::open`, you will need to create the LSM database directory if it does not exist. To flush a memtable to the disk, we will need to do three things:

- Select a memtable to flush.
- Create an SST file corresponding to a memtable.
- Remove the memtable from the immutable memtable list and add the SST file to L0 SSTs.

We have not explained what is L0 (level-0) SSTs for now. In general, they are the set of SSTs files directly created as a result of memtable flush. In week 1 of this course, we will only have L0 SSTs on the disk. We will dive into how to organize them efficiently using leveled or tiered structure on the disk in week 2.

Note that creating an SST file is a compute-heavy and a costly operation. Again, we do not want to hold the `state` read/write lock for a long time, as it might block other operations and create huge latency spikes in the LSM operations. Also, we use the `state_lock` mutex to serialize state modification operations in the LSM tree. In this task, you will need to think carefully how to use these locks to make the LSM state modification race-condition free while minimizing critical sections.

We do not have concurrent test cases and you will need to think carefully about your implementation. Also, remember that the last memtable in the immutable memtable list is the earliest one, and is the one that you should flush.

► Spoilers: Flush L0 Pseudo Code

Task 2: Flush Trigger

In this task, you will need to modify:

```
src/lsm_storage.rs
src/compact.rs
```

When the number of memtables (immutable + mutable) in memory exceeds the `num_memtable_limit` in LSM storage options, you should flush the earliest memtable to the disk. This is done by a flush thread in the background. The flush thread will be started with the `MiniLSM` structure. We have already implemented necessary code to start the thread and properly stop the thread.

In this task, you will need to implement `LsmStorageInner::trigger_flush` in `compact.rs`, and `MiniLsm::close` in `lsm_storage.rs`. `trigger_flush` will be executed every 50 milliseconds. If the number of memtables exceed the limit, you should call `force_flush_next_imm_memtable` to flush a memtable. When the user calls the `close` function, you should wait until the flush thread (and the compaction thread in week 2) to finish.

Task 3: Filter the SSTs

Now that you have a fully working storage engine, and you can use the `mini-lsm-cli` to interact with your storage engine.

```
cargo run --bin mini-lsm-cli -- --compaction none
```

And then,

```
fill 1000 3000
get 2333
flush
fill 1000 3000
get 2333
flush
get 2333
scan 2000 2333
```

If you fill more data, you can see your flush thread working and automatically flushing the L0 SSTs without using the `flush` command.

And lastly, let us implement a simple optimization on filtering the SSTs before we end this week. Based on the key range that the user provides, we can easily filter out some SSTs that do not contain the key range, so that we do not need to read them in the merge iterator.

In this task, you will need to modify:

```
src/lsm_storage.rs
src/iterators/*
src/lsm_iterator.rs
```

You will need to change your read path functions to skip the SSTs that is impossible to contain the key/key range. You will need to implement `num_active_iterators` for your iterators so that the test cases can do the check on whether your implementation is correct or not. For `MergeIterator` and `TwoMergeIterator`, it is the sum of `num_active_iterators` of all children iterators. Note that if you did not modify the fields in the starter code of `MergeIterator`, remember to also take `MergeIterator::current` into account. For `LsmIterator` and `FusedIterator`, simply return the number of active iterators from the inner iterator.

You can implement helper functions like `range_overlap` and `key_within` to simplify your code.

Test Your Understanding

- What happens if a user requests to delete a key twice?
- How much memory (or number of blocks) will be loaded into memory at the same time when the iterator is initialized?
- Some crazy users want to *fork* their LSM tree. They want to start the engine to ingest some data, and then fork it, so that they get two identical dataset and then operate on

them separately. An easy but not efficient way to implement is to simply copy all SSTs and the in-memory structures to a new directory and start the engine. However, note that we never modify the on-disk files, and we can actually reuse the SST files from the parent engine. How do you think you can implement this fork functionality efficiently without copying data? (Check out [Neon Branching](#)).

- Imagine you are building a multi-tenant LSM system where you host 10k databases on a single 128GB memory machine. The memtable size limit is set to 256MB. How much memory for memtable do you need for this setup?
 - Obviously, you don't have enough memory for all these memtables. Assume each user still has their own memtable, how can you design the memtable flush policy to make it work? Does it make sense to make all these users share the same memtable (i.e., by encoding a tenant ID as the key prefix)?

We do not provide reference answers to the questions, and feel free to discuss about them in the Discord community.

Bonus Tasks

- **Implement Write/L0 Stall.** When the number of memtables exceed the maximum number too much, you can stop users from writing to the storage engine. You may also implement write stall for L0 tables in week 2 after you have implemented compactions.
- **Prefix Scan.** You may filter more SSTs by implementing the prefix scan interface and using the prefix information.

Your feedback is greatly appreciated. Welcome to join our [Discord Community](#).

Found an issue? Create an issue / pull request on github.com/skyzh/mini-lsm.

mini-lsm-book © 2022-2025 by Alex Chi Z is licensed under CC BY-NC-SA 4.0.